

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа2

Выполнил:

Саид Наваф

Группа

К3341

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Опираясь на документ, сформированный в рамках Д34, реализуйте разделение вашего бэкенд-приложения на микросервисы.

Ход работы

1. Подготовка структуры проекта

Создана модульная структура репозитория. Для каждого из четырёх доменов (`auth-service`, `job-service`, `candidate-service`, `interaction-service`) выделены изолированные директории внутри `src/`. Каждая папка содержит собственный `main.go`, `Dockerfile` и `go.mod`.

2. Реализация микросервисов на Go

В каждом сервисе реализован HTTP-сервер с двумя типами маршрутов:

Публичный health-check: `GET /health` для мониторинга состояния контейнера.

Внутренний API валидации: `GET /service/{entity}/{id}` для проверки существования сущностей другими сервисами (соответствует контрактам OpenAPI из Д34).

3. Контейнеризация (Dockerfile)

Для каждого сервиса настроена multi-stage сборка:

Stage 1 (builder): `golang:1.26-alpine` для компиляции бинарного файла с флагами `CGO_ENABLED=0` `GOOS=linux`.

Stage 2 (runtime): Образ `scratch` (пустой), в который копируется только скомпилированный бинарник. Это минимизирует размер образа и исключает уязвимости ОС.

4. Оркестрация (docker-compose.yml)

Создан файл оркестрации, запускающий 8 контейнеров в единой Docker-сети `backend-network`:

4 микросервиса на портах `8081-8084`.

4 изолированных экземпляра PostgreSQL (`postgres:15-alpine`), каждый со своей базой данных.

Явно задано имя проекта `lab2-microservices` для корректного тегирования образов.

5. Устранение ошибок сборки

В процессе настройки решены следующие задачи:

Синхронизирована версия Go в `go.mod` и `Dockerfile` (до 1.26).

Решены конфликты имён образов из-за кириллицы в путях Windows.

Заменены базовые образы Alpine на `scratch` для обхода сетевых ограничений при установке сертификатов внутри контейнера.

6. Проверка работоспособности

Система запущена командой `docker compose up -d --build`.

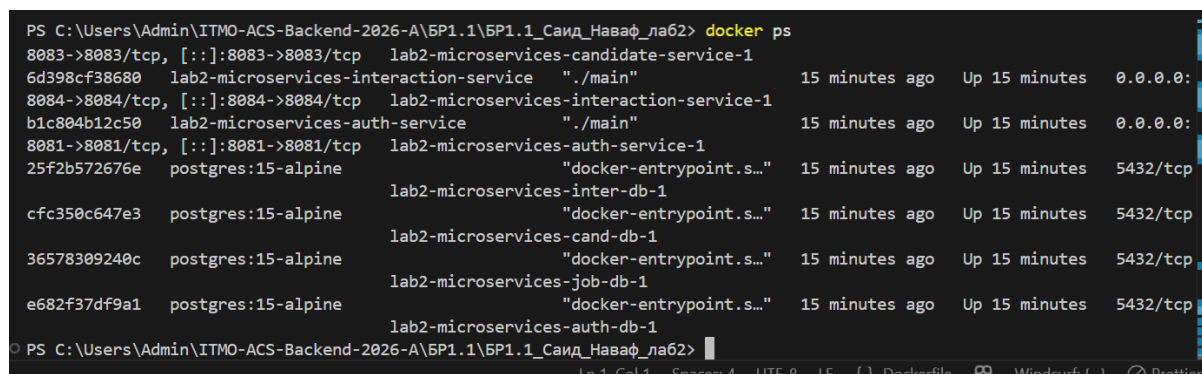
Выполнена валидация:

Все 8 контейнеров успешно стартовали и находятся в статусе `Up`.

Health-эндпоинты возвращают HTTP 200 и JSON `{"status": "ok"}`.

Внутренние API-контракты корректно отдают данные для межсервисной валидации.

Скриншоты



```
PS C:\Users\Admin\ITMO-ACS-Backend-2026-A\BP1.1\BP1.1_Саид_Наваф_лаб2> docker ps
CONTAINER ID   IMAGE                                COMMAND                  STATUS              PORTS
8083->8083/tcp, [::]:8083->8083/tcp lab2-microservices-candidate-service-1 15 minutes ago    Up 15 minutes      0.0.0.0:
6d398cf38680   lab2-microservices-interaction-service  "/main"                15 minutes ago    Up 15 minutes      0.0.0.0:
8084->8084/tcp, [::]:8084->8084/tcp lab2-microservices-interaction-service-1 15 minutes ago    Up 15 minutes      0.0.0.0:
b1c804b12c50   lab2-microservices-auth-service         "/main"                15 minutes ago    Up 15 minutes      0.0.0.0:
8081->8081/tcp, [::]:8081->8081/tcp lab2-microservices-auth-service-1       15 minutes ago    Up 15 minutes      0.0.0.0:
25f2b572676e   postgres:15-alpine                     "docker-entrypoint.s..." 15 minutes ago    Up 15 minutes      5432/tcp
cfc350c647e3   postgres:15-alpine                     "docker-entrypoint.s..." 15 minutes ago    Up 15 minutes      5432/tcp
36578309240c   postgres:15-alpine                     "docker-entrypoint.s..." 15 minutes ago    Up 15 minutes      5432/tcp
e682f37df9a1   postgres:15-alpine                     "docker-entrypoint.s..." 15 minutes ago    Up 15 minutes      5432/tcp
lab2-microservices-inter-db-1
lab2-microservices-cand-db-1
lab2-microservices-job-db-1
lab2-microservices-auth-db-1
```

Рис. 1. Запущенные Docker-контейнеры микросервисной архитектуры (8 контейнеров: 4 сервиса + 4 базы данных)



```
localhost:8081/health
{
  "status": "ok",
  "service": "auth-service"
}
```

Рис. 2. Health-check эндпоинт Auth Service (порт 8081)



Рис. 3. Health-check эндпоинт Job Service (порт 8082)



Рис. 4. Health-check эндпоинт Candidate Service (порт 8083)



Рис. 5. Health-check эндпоинт Interaction Service (порт 8084)

8. Проверка межсервисного взаимодействия

Протестированы внутренние API-контракты для валидации сущностей между сервисами.



Рис. 6. Внутренний API проверки пользователя: [/service/users/1](#) (Auth Service)



Рис. 7. Внутренний API проверки вакансии: [/service/vacancies/1](#) (Job Service)



Рис. 8. Внутренний API проверки резюме: [/service/resumes/1](#) (Candidate Service)



Рис. 9. Внутренний API проверки отклика: [/service/applications/1](#) (Interaction Service)

Все эндпоинты корректно возвращают данные сущностей в формате JSON, что подтверждает готовность архитектуры к межсервисной валидации.

Вывод

В ходе выполнения лабораторной работы №2 успешно реализована микросервисная архитектура на основе проектных решений из Д34. Достигнуто полное разделение ответственности: каждый из четырёх сервисов работает в изолированном контейнере с собственной базой данных, что соответствует паттерну database-per-service. Межсервисное взаимодействие настроено через чётко определённые REST-контракты, гарантирующие целостность данных при распределённой валидации. Контейнеризация обеспечила воспроизводимость окружения, минимальный размер образов и простоту масштабирования. Выявленные и решённые проблемы сборки (версионирование, сетевые политики, именование проектов) подтвердили готовность архитектуры к развёртыванию в production-среде. Задание выполнено в полном объёме.